



Estatística Computacional com R (PPGBC/UFPA e PPGCTIF/UFOPA)



Introdução ao R

(Instalação, Sintaxe e Projetos)



Prof. Dr. Deivison Venicio Souza

Universidade Federal do Pará (UFPA)

Faculdade de Engenharia Florestal

Laboratório de Manejo Florestal, Tecnologias e Comunidades Amazônicas

E-mail: deivisonvs@ufpa.br

1ª versão: 14/outubro/2022

(Atualizado em: 28/junho/2026)



PPGCTIF



Objetivos



Ao final do **Módulo 1 - Introdução ao R-base** espera-se que os discentes possam alcançar conhecimentos para...

- Compreender a importância da linguagem R e do RStudio;
- Conhecer a sintaxe básica da linguagem R;
- Criar objetos, usar atribuição, comentários e funções;
- Utilizar a documentação integrada do R;
- Organizar análises por meio de Projetos no RStudio;
- Compreender documentação e reprodutibilidade; e
- Aplicar boas práticas em projetos R.



Conteúdo



Parte 1 – Conhecendo o R e o RStudio

- 1- A linguagem R: Origem
- 2 - A linguagem R: multiplas facetas
- 3 - Por que utilizar o R?
- 4 - RGui: download, instalação e interface
- 5 - RStudio: download, instalação e interface

Parte 2 – Sintaxe da Linguagem R

- 1 - R é uma Linguagem de Expressão
- 2 - R é Case Sensitive
- 3 - Operador de Atribuição
- 4 - Comentários
- 5 - Funções
- 6 - Ajuda e Documentação



Conteúdo



Parte 3 – Projetos no RStudio

- 1 - O que é um Projeto (.Rproj)?
- 2 - Por que utilizar Projetos?
- 3 - Criando um Projeto no RStudio
- 4 - Estrutura Recomendada
- 5 - Documentando um projeto (README.md)
- 6 - Reprodutibilidade
- 7 - Registrando o Ambiente Computacional
- 8 - Boas Práticas



Módulo 1

Introdução ao R básico

Parte 1 - Conhecendo o R e o IDE Rstudio



Linguagem de Programação R - Origem



- Linguagem de programação **livre, gratuita** e de **código aberto**;
- Desenvolvida em 1993 por **Ross Ihaka** e **Robert Gentleman**;
- Criada na Universidade de **Auckland** (Nova Zelândia);
- Baseada na **linguagem S**; e
- Amplamente usada em estatística, ciência de dados e pesquisa científica.



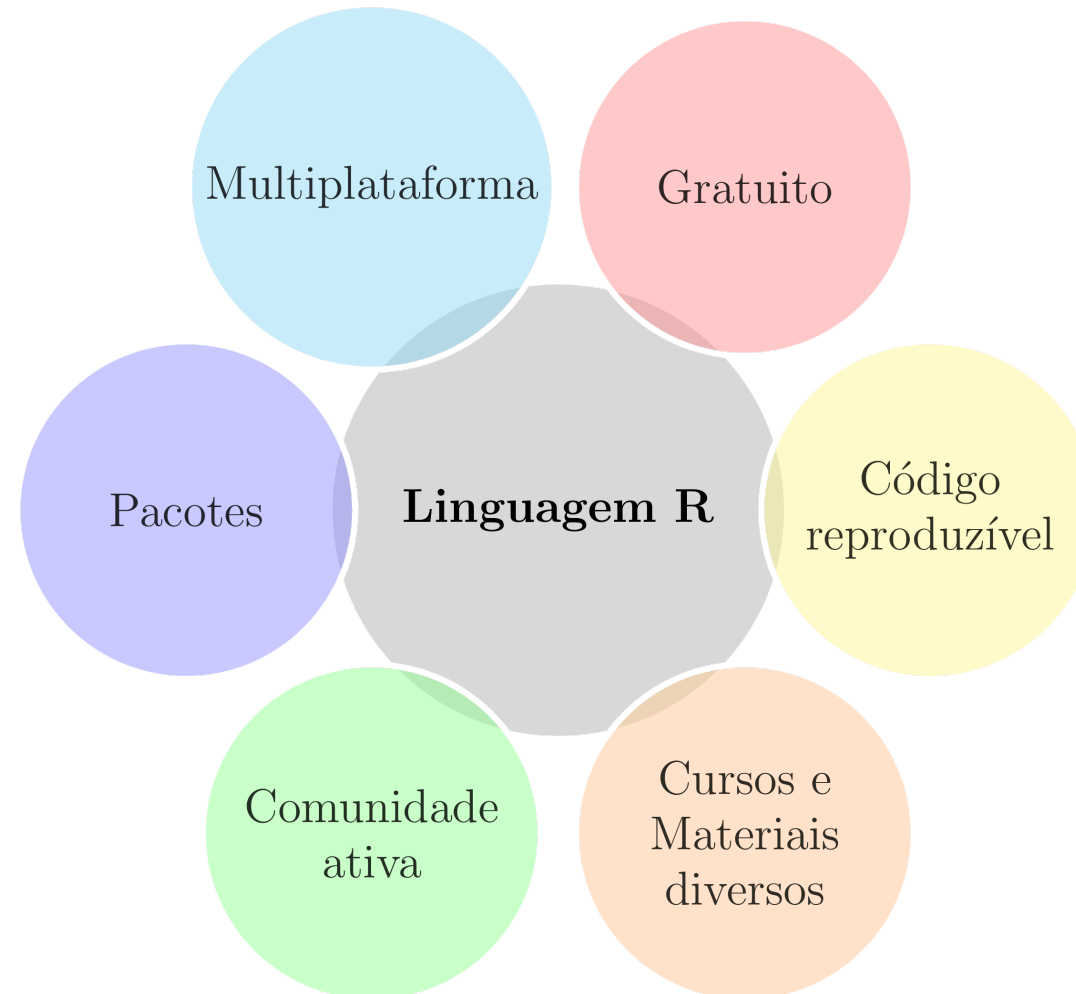
Linguagem de Programação R - Múltiplas Facetas



-  **Análise estatística**
-  **Manipulação de dados**
-  **Visualização gráfica**
-  **Aplicações web e dashboards**
-  **Relatórios dinâmicos**
-  **Apresentações profissionais**



Por que usar a linguagem R?



RGui - Download e instalação



1º Passo: Acesse o site oficial do R:

 <https://cloud.r-project.org/>

 Seleciona automaticamente o melhor servidor para download.

CRAN (Comprehensive R Archive Network)

- Repositório oficial da linguagem R;
- Disponibiliza instaladores para Windows, Linux e macOS;
- Hospeda milhares de pacotes desenvolvidos pela comunidade;
- Oferece documentação e materiais de apoio.



RGui - Download e instalação



2º Passo: Na página inicial, selecione o sistema operacional do seu computador:

1. Download R for Windows;
2. Download R for Linux; ou
3. Download R for MacOS.

3º Passo: Clique em **Install R for the first time**.

4º Passo: Faça o download da versão mais recente do R. (R-4.6.0 for Windows)

5º Passo: Execute o arquivo baixado. Siga as etapas do instalador, mantendo as configurações padrão.

✓ **Pronto! O R foi instalado com sucesso e está pronto para uso.**

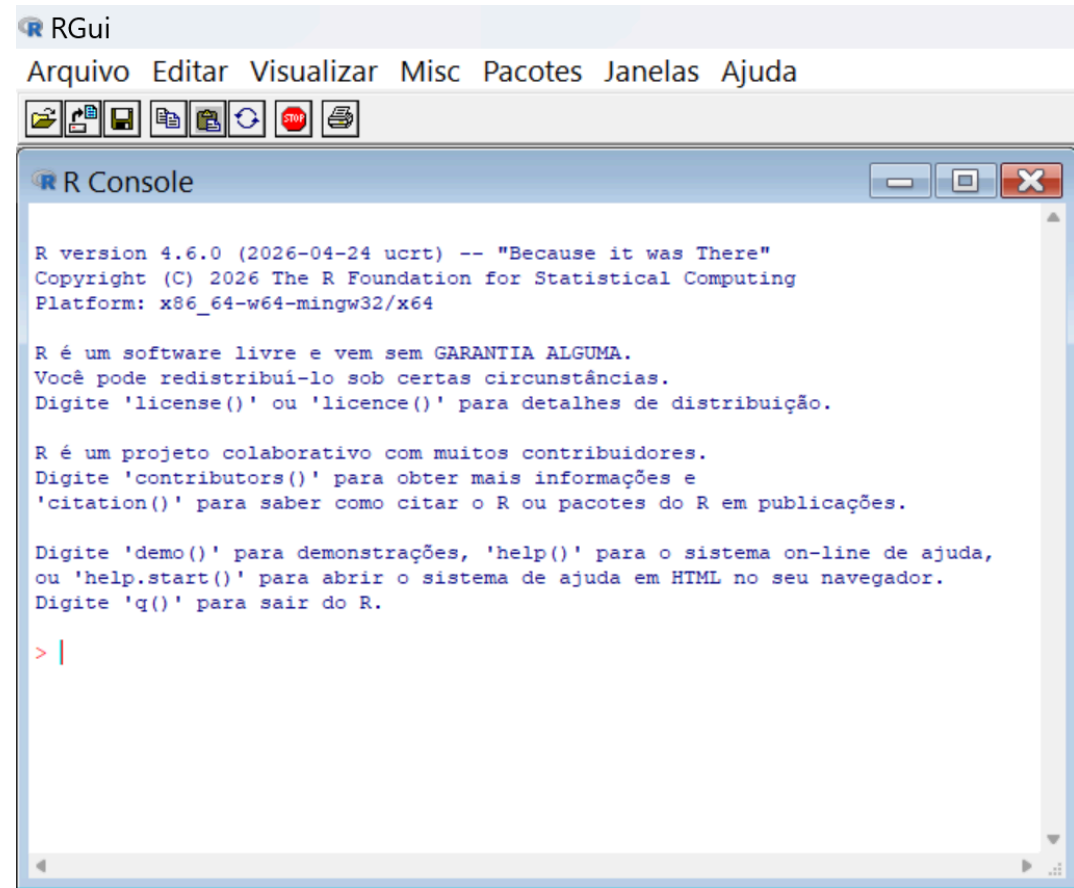


RGui - Interface

- Ao iniciar o RGui, é exibido o **R Console**.
- O símbolo **>** representa o prompt de comando.

Experimente:

`getwd()` `help()` `q()`



```
RGui
Arquivo  Editar  Visualizar  Misc  Pacotes  Janelas  Ajuda

R Console

R version 4.6.0 (2026-04-24 ucrt) -- "Because it was There"
Copyright (C) 2026 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64

R é um software livre e vem sem GARANTIA ALGUMA.
Você pode redistribuí-lo sob certas circunstâncias.
Digite 'license()' ou 'licence()' para detalhes de distribuição.

R é um projeto colaborativo com muitos contribuidores.
Digite 'contributors()' para obter mais informações e
'citation()' para saber como citar o R ou pacotes do R em publicações.

Digite 'demo()' para demonstrações, 'help()' para o sistema on-line de ajuda,
ou 'help.start()' para abrir o sistema de ajuda em HTML no seu navegador.
Digite 'q()' para sair do R.

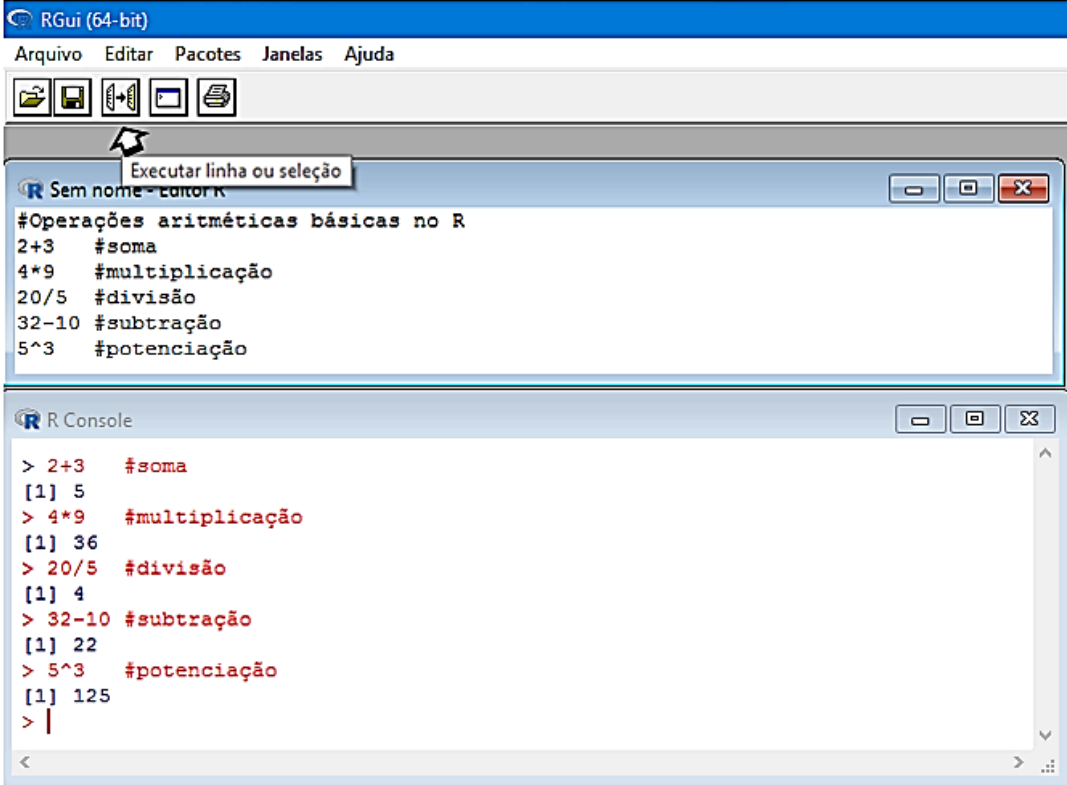
> |
```

R Console



RGui - R Editor

- O RGui possui um **R Editor** para criação e edição de scripts.
- Acesse: **Arquivo** → **Novo script** ou **Arquivo** → **Abrir script**.
- O código é executado no **R Console** usando **Ctrl + R**.



The screenshot displays the RGui (64-bit) application window. The top menu bar includes 'Arquivo', 'Editar', 'Pacotes', 'Janelas', and 'Ajuda'. Below the menu is a toolbar with icons for file operations. The main window is divided into two panes. The top pane, titled 'Sem nome - Editor', contains a script with the following text: '#Operações aritméticas básicas no R', '2+3 #soma', '4*9 #multiplicação', '20/5 #divisão', '32-10 #subtração', and '5^3 #potenciação'. A tooltip 'Executar linha ou seleção' is visible over the script. The bottom pane, titled 'R Console', shows the execution of the script, with the following output: '> 2+3 #soma', '[1] 5', '> 4*9 #multiplicação', '[1] 36', '> 20/5 #divisão', '[1] 4', '> 32-10 #subtração', '[1] 22', '> 5^3 #potenciação', '[1] 125', and '> |'.

R editor



IDE RStudio - download e instalação

👉 O **RStudio** é um ambiente integrado de desenvolvimento (IDE) amplamente utilizado por usuários da linguagem R.

1º Passo: Acessar o site oficial: <https://www.rstudio.com>;

2º Passo: Products → RStudio;

3º Passo: Selecionar **RStudio Desktop**;

4º Passo: Escolher a versão **Open Source** e clicar em **Download RStudio Desktop**;

5º Passo: Executar o instalador baixado e concluir a instalação.

💡 O R deve estar instalado previamente para o funcionamento do RStudio.



IDE RStudio - Interface



The screenshot displays the RStudio IDE interface with the following components:

- Editor:** Contains R code for loading the `ggplot2` package, loading the `iris` dataset, and creating a scatter plot of `Sepal.Length` vs `Petal.Length` colored by `Species`.
- Environment:** Shows the `Global Environment` with the `iris` dataset loaded, containing 150 observations of 5 variables.
- Console:** Displays the executed R commands and their output.
- Output:** Shows the resulting scatter plot of `Sepal.Length` vs `Petal.Length`, colored by `Species` (setosa, versicolor, virginica).



IDE RStudio - Interface (Painéis)



Editor: Painel de desenvolvimento dos códigos R.

Environment: Painel onde aparecerão todos os objetos criados no R.

Console: Painel para rodar os códigos R e receber outputs.

Plots: Painel de saída gráfica.

History: Painel que mostra um histórico dos comandos executados na sessão corrente.

Help: Painel que mostra a documentação de funções de pacotes, quando solicitada.

Files: Painel para identificar arquivos no diretório de trabalho.

Packages: Painel que mostra os pacotes instalados. É possível identificar os pacotes carregados na sessão corrente.



Módulo 1

Introdução ao R-Base

Parte 2 - Sintaxe da linguagem R



R é uma Linguagem de Expressão



👉 Em R, toda instrução é avaliada para produzir um **valor** ou um **objeto**.

- **Objetos:** armazenam dados, funções, modelos, gráficos e outros resultados.
- **Avaliação:** cada comando é processado e retorna um resultado.
- **Concisão:** Permite combinar operações de forma simples e eficiente.

💡 Um **objeto** é qualquer elemento armazenado na memória do R.

Exemplo: `mean(c(10, 20, 30))` cria um vetor e retorna sua média.



R é Case Sensitive

Conceito

👉 Letras **maiúsculas** e **minúsculas** possuem significados diferentes.

`a` $\neq$ `A`

`altura` $\neq$ `Altura`

⚠️ Diferenças de capitalização podem gerar erros do tipo *"object not found"*.

Recomendação

💡 Adote o padrão **snake_case**:

- Apenas letras minúsculas;
- Palavras separadas por `_`.

`altura_arvore`

`volume_total`

`media_dap`



Operador de Atribuição



👉 Em R, utiliza-se `<-` para atribuição de valores.

`<-` → Atribuição

`==` → Comparação

```
# atribuição (cria/modifica um objeto)
dap <- 20
```

```
# comparação (retorna TRUE ou FALSE)
dap == 20
```

💡 `<-` atribui valores; `==` faz perguntas.



Comentários



👉 Comentários servem para **documentar e explicar o código**.

👉 O R ignora comentários durante a execução.

```
# Armazena os DAPs das árvores  
dap <- c(12.5, 15.8, 18.2, 20.1)  
  
# Calcula a média  
mean(dap)
```

O símbolo # indica um comentário.

💡 Um código bem comentado hoje economiza tempo no futuro!



Funções



👉 Grande parte das tarefas em R é realizada por meio de funções.

👉 Funções executam tarefas específicas.

Estrutura geral:

nome_da_funcao(argumentos)

Exemplo

```
sqrt(25)
## [1] 5
sum(c(10, 20, 30))
## [1] 60
```

💡 Funções recebem **argumentos** e retornam **resultados**.



Funções - Alguns Exemplos



Função	Descrição
<code>sqrt()</code>	Raiz quadrada
<code>abs()</code>	Valor absoluto
<code>sin(); cos(); tan()</code>	Funções trigonométricas
<code>asin(); acos(); atan()</code>	Funções trigonométricas inversas
<code>exp()</code>	Exponencial
<code>log10()</code>	Logarítmo na base 10
<code>log()</code>	Logarítmo natural
<code>factorial</code>	Fatorial



Funções - Alguns Exemplos



Funções	Ação
q()	Fechar o programa
rm(nome do objeto)	Remover um objeto qualquer
ls()	Listar os objetos na janela de trabalho atual
help(nome da função) ou ?nome da função	Solicitar ajuda sobre o uso de uma função
save.image()	Salvar
Ctrl + L	Limpar a tela do R console
history(max.show, nrow = 3)	Listar os últimos 3 comandos executados
getwd()	Mostrar o diretório de trabalho
setwd("diretório desejado")	Mudar o diretório de trabalho
install.packages("nome do pacote")	Instalar um pacote específico



Funções - Alguns Exemplos



Funções	Ação
library("nome do pacote")	Carregar um pacote específico
dir()	Lista os arquivos existentes no diretório
getOption("OutDec")	Verificar o separador decimal definido
options("OutDec=")	Mudar o separador decimal para vírgula
round(5.9845, digits=2)	Função para arredondamento de casas decimais
data()	Lista de conjuntos de dados disponíveis no R
?nomedodataset	Obter informações detalhadas sobre um conjunto de dado
class(nome do objeto)	Verifica a classe de um objeto específico
search()	Lista todos os pacotes carregados



Ajuda e Documentação



👉 O R possui um sistema de ajuda integrado.

Comandos úteis:

?funcao

help(funcao)

example(funcao)

Exemplos

```
?mean
```

```
help(mean)
```

```
example(mean)
```

💡 Consulte a documentação sempre que tiver dúvidas sobre uma função.



O que aprendemos?



- ✓ R é uma linguagem de expressão
- ✓ R é case sensitive
- ✓ Utilizamos `<-` para atribuição
- ✓ Comentários iniciam com `#`
- ✓ Funções seguem o padrão `funcao(argumentos)`
- ✓ A documentação pode ser acessada com ?



Módulo 1

Introdução ao R-Base

Parte 3 - Projetos no RStudio



O que é um Projeto (.Rproj)?



👉 Um projeto reúne todos os arquivos de uma análise em um único local.

Dados + Scripts + Resultados + Relatórios

Um projeto pode conter:

- Dados;
- Scripts em R;
- Figuras;
- Tabelas;
- Relatórios;
- Apresentações.



Por que utilizar Projetos?



Sem Projeto

- ✗ Arquivos espalhados
- ✗ Uso de caminhos absolutos (`setwd()`)
- ✗ Código menos reproduzível
- ✗ Maior dificuldade para compartilhar análises

Com Projeto

- ✓ Arquivos organizados
- ✓ Uso de caminhos relativos
- ✓ Diretório raiz definido pelo `.Rproj`
- ✓ Código mais reproduzível
- ✓ Compartilhamento facilitado

💡 Projetos tornam as análises mais organizadas, reproduzíveis e fáceis de manter.



Criando um Projeto no RStudio



Passos

1. **File** → **New Project**
2. **New Directory**
3. **New Project**
4. **Directory name:** Forneça um nome
5. **Browser:** Escolha o local
6. Clique em **Create Project**



O RStudio criará automaticamente o arquivo `.Rproj`, que definirá o diretório raiz do projeto.



Estrutura Recomendada



👉 **Boa prática:** [Seja organizado!](#) Mantenha os dados, códigos e resultados em diretórios separados.



Diretório	Finalidade
data	Dados utilizados na análise
script	Scripts em R
figure	Figuras e gráficos
table	Tabelas geradas
reports	Relatórios e documentos finais



Documentando o Projeto: README.md



👉 **README.md**: arquivo de documentação que descreve o projeto e orienta sua utilização.

```

📁 Projeto
├── 📄 README.md
├── 📁 data
├── 📁 script
├── 📁 output
│   ├── 📁 figure
│   ├── 📁 table
│   └── 📁 reports
└── ® Projeto.Rproj (diretório raiz)
```

Informações básicas

- Título do projeto;
- Objetivo(s);
- Autor(es);
- Estrutura de diretórios;
- Dados utilizados;
- Principais scripts; e
- Data da última atualização.

💡 Não basta organizar o projeto com **.Rproj**, é importante documentá-lo com **README.md**.



Criando um README.md



Como criar?

1. **File** → **New File** → **Text File**

ou

1. **File** → **New File** → **R Markdown**

Salvar como:

README.md

ou

README.Rmd

Onde salvar?

```
Projeto
├── README.md
├── data
├── script
├── output
│   ├── figure
│   ├── table
│   └── reports
└── ® Projeto.Rproj
```



O **README.md** é o primeiro arquivo que deve ser lido para compreender o projeto.



Exemplo de README.md

Biomassa de Cacau

🎯 Objetivo

Ajustar modelos para estimativa de biomassa de cacaueiros.

👤 Autor

Deivison Souza

📊 Dados

data/biomassa.csv

📄 Scripts

01_importacao.R

02_analise.R

🕒 Atualização

02/06/2026

README.md

1 # Biomassa de Cacau

2

3 ## 🎯 Objetivo

4 Ajustar modelos para estimativa
5 de biomassa de cacaueiros.

6

7 ## 👤 Autor

8 Deivison Souza

9

10 ## 📊 Dados

11 data/biomassa.csv

12

13 ## 📄 Scripts

14 01_importacao.R

15 02_analise.R

16

🕒 Atualização

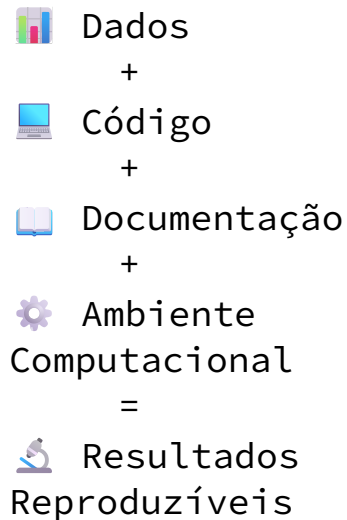
02/06/2026



Reprodutibilidade



Componentes



Boas Práticas

- ✓ Utilizar Projetos (.Rproj)
- ✓ Utilizar caminhos relativos
- ✓ Manter os dados brutos intactos
- ✓ Criar um README .md
- ✓ Registrar versões dos pacotes

💡 Uma análise é **reproduzível** quando produz os **mesmos resultados** a partir dos **mesmos dados** e **procedimentos**.



Registrando o Ambiente Computacional

O que é?

👉 O pacote **renv** permite registrar o ambiente computacional usado em um projeto.

Por que é importante?

👉 O mesmo código pode produzir resultados diferentes em computadores distintos.

O que é registrado?

- ✓ Versão do R
- ✓ Pacotes e versões
- ✓ Dependências do projeto

💻 Computador A
R 4.6.0
dplyr 1.1.4

💻 Computador B
R 5.0.0
dplyr 2.0.0

💡 Diferenças nas versões do R e dos pacotes podem afetar a reprodutibilidade das análises.

Utilizando o renv

Como criar?

1. Instalar o pacote:

```
install.packages("renv")
```

1. Inicializar o projeto:

```
renv::init()
```

O arquivo criado será:

renv.lock

Onde salvar?

```
Projeto
├── README.md
├── renv.lock
├── data
├── script
├── output
│   ├── figure
│   ├── table
│   └── reports
└── ® Projeto.Rproj
```

💡 O arquivo `renv.lock` registra o ambiente computacional do projeto, aumentando sua reprodutibilidade.

Boas Práticas



Organização

- ✓ Um projeto para cada análise (.Rproj)
- ✓ Dados, scripts e resultados separados
- ✓ Utilizar caminhos relativos
- ✓ Evitar o uso de `setwd()`



Reprodutibilidade

- ✓ Documentar o projeto com `README.md`
- ✓ Preservar os dados brutos
- ✓ Registrar o ambiente computacional
- ✓ Compartilhar dados e scripts

 Um projeto bem estruturado deve ser **compreendido, executado e reproduzido** por qualquer pessoa.



Dúvidas?



Mensagem

Dominar a **sintaxe do R** e utilizar **Projetos no RStudio** são passos fundamentais para criar análises **organizadas e reproduzíveis**.



Próximo passo

Explorar as principais **estruturas de dados** em R e aprender como armazenar informações de forma eficiente.



Perguntas, comentários ou sugestões?

Obrigado!



@lmftca_ufpa



<https://www.lmftca.com.br/>

